Transformer 横纵分析报告

研究对象：Transformer 深度学习架构

类型：AI 模型 / 技术范式

方法论：横纵分析法（Horizontal-Vertical Analysis）

撰写时间：2026-04-29

目录

1. 一句话定义
2. 技术背景：Transformer 前置知识

- 2.1 前 Transformer 时代 · 2.2 注意力 QKV · 2.3 多头注意力 · 2.4 位置编码
- 2.5 FFN 记忆库 · 2.6 为什么能 scale · 2.7 训练与推理 · 2.8 分词 · 2.9 图谱速查

1. 纵向分析：从一场豪赌到统治世界

- 2017 诞生 → 2018 分歧 → 2019-2020 爆发 → 2021-2022 收敛 → 2023 开源反击 → 2024 MoE 奇迹 → 2025-2026 新常态

1. 横向分析：九年后，Transformer 站在什么位置

- 六大派系 · 注意力三方会战 · MoE 四大分歧 · Claude 为何坚持 Dense · 三种 Scaling

1. 横纵交汇洞察

- 历史的必然与偶然 · MoE 视角重审 · 三个未来剧本

1. 实战：面试题、场景题、应用题

- 10 道面试题 · 5 道场景题 · 3 道应用题

1. 信息来源

一、一句话定义

Transformer 是一种完全基于注意力机制、彻底抛弃了循环神经网络的序列建模架构。2017 年诞生时只是机器翻译的一个实验方案，如今它的变体驱动了全球几乎所有大语言模型——GPT-4、Claude、Gemini、DeepSeek——以及视觉（ViT）、语音、蛋白质折叠（AlphaFold）等 AI 前沿。

它解决了一个根本矛盾：**RNN 串行计算的龟速 vs 长序列建模的刚需**。解决方式是自注意力——每个 token 同时看到序列中所有其他 token，训练完全并行。

代价是 $O(n^2)$ 的注意力复杂度。九年来，几乎所有的架构创新都在跟这个代价搏斗。

二、技术背景：Transformer 前置知识

要理解 Transformer 为什么是革命，得先理解它革了什么命。

2.1 前 Transformer 时代：RNN 帝国的黄昏

2017 年之前，序列建模是 RNN 的天下。一条句子进来，RNN 从左到右逐个 token 吞下，每一步输出一个隐状态 h_t，传给下一步。LSTM 和 GRU 给 RNN 加了门控机制，缓解了梯度消失——你能记住 20 个 token 之前的"主语是单数"这种信息了。

但有个绕不过去的硬伤：**串行计算**。

要算 h_{100}，必须先把前面的 99 步都算完。GPU 几千个核心，用不上。训练一条长句子，要等 CPU 时代的逻辑在 GPU 上慢慢挪。更致命的是，虽然 LSTM 能记住 20 步前，到 100 步、200 步，该忘还是忘——反向传播路径太长，梯度信号被洗成噪声。

Seq2Seq 架构（Sutskever et al., 2014）给这个问题打了个补丁：用一个定长向量 c 代表整个源句，decoder 从 c 解码。问题显而易见——"定长向量压缩整句"本身就是信息瓶颈。句子一长，前半句的信息被挤掉了。

注意力机制（Bahdanau et al., 2014）是第一个真正的突破：不再依赖一个定长向量，decoder 每一步都**动态搜索**整个源句，自己去挑"现在该看哪些词"。这是关键的思想跃迁——从"被动接收一个压缩表示"变成了"主动检索"。这个"软寻址"的视角，后来成了理解 Transformer 的核心隐喻。

2.2 注意力：Query、Key、Value 到底是什么

自注意力（Self-Attention）是 Transformer 的心脏。一句话说明白：**让每个 token 去问其他 token："你跟我什么关系？"然后根据关系强弱，加权汇总其他 token 的信息**。

三个矩阵 W_Q、W_K、W_V 做的是同一件事的不同阶段：

- **Query（查询）**：当前 token 在问什么——"我是动词 'eat'，我需要知道谁在吃、吃了什么"
- **Key（键）**：每个 token 提供什么信息标签——"我是名词，我可以当主语"
- **Value（值）**：每个 token 携带的实质信息——"我的具体语义是 'cat'"

计算过程：

```
Attention(Q, K, V) = softmax(QK^T / √d_k) × V
```

QK^T 算出注意力分数矩阵——每个位置对其他所有位置的"关注度"。除以 √d_k 是因为维度一大，点积值爆炸，softmax 会进入梯度几乎为零的区域。softmax 归一化成概率分布。乘以 V，加权汇总。这一层的输出是"所有 token 的信息按关注度重新调配的结果"。

这个视角跟检索系统一模一样：Query 是搜索词，Key 是文档索引，Value 是文档内容。跟 MoE 的 router 也很像——MoE 用门控网络决定"每个 token 去哪几个 expert"，Transformer 用 QK 相似度决定"每个 token 关注哪几个 token"。都是**软路由**，只是路由的对象不同：Transformer 路由信息，MoE 路由计算。

⚡ **一张图理解 QKV 的计算过程（小白视角）**

想象你要翻译这句话："The cat sat on the mat because it was tired."

当模型处理"it"这个 token 时：

```
"it" 的 Query = "我正在找一个单数名词，我可能是代词指代"
所有 token 的 Keys = ["The": 冠词, "cat": 单数名词, "sat": 动词, ...]
"cat" 的 Key 跟 "it" 的 Query 匹配度最高 → Attention 分数最大
"cat" 的 Value = "猫" → 被大部分权重传递给 "it"
```

结果：模型"知道"了 it 指的是 cat，而不是 mat。

Cross-Attention vs Self-Attention

上面说的是 Self-Attention（自己看自己），Encoder-Decoder 架构里还有一种 Cross-Attention：

- Self-Attention：序列内部的 token 互看。编码器能看整句、解码器只能看已生成部分
- Cross-Attention：Decoder 的 Query 去检索 Encoder 的所有 Key/Value，问"源句里哪个词跟我要生成的下一个词最相关？"
- 这跟 MoE 的门控网络逻辑一致——都是「根据当前需要，从候选池里挑最相关的信息」

Decoder-only 架构（GPT 系）之所以能成为主流，正是因为**把 Cross-Attention 省掉了**——所有知识都包装在 Self-Attention + FFN 里，架构更统一，推理也更简洁。

2.3 多头注意力：为什么需要八个头

单头注意力的表达能力有限——一个 token 在一次前向中只能形成一种"关系模式"。但实际上，一个 token 可能需要同时关注"语法结构"（动词接什么介词）和"语义角色"（谁是施事者）。

多头注意力的做法：把 Q、K、V 分别投影到多个低维子空间（比如 8 个头、每个头 d_k = 512/8 = 64），每个头独立计算注意力，最后拼接。

```
MultiHead(Q, K, V) = Concat(head_1, ..., head_h) × W_O
```

效果是，不同头确实学会了关注不同的关系模式。有的头负责相邻词之间的句法依赖，有的头负责长距离语义关联，有的头专门看标点符号。这不是设计出来的——是训练出来的涌现行为。

从 MoE 的视角看，多头注意力很像"在注意力维度上做了细粒度专家"——每个头是一个专门的关系检测器，它们并行工作、最后融合。这也解释了为什么后来 DeepSeek 的 MLA 能用低秩压缩减少 KV cache：多个头之间的信息有大量

重叠，可以被压缩进一个更小的潜空间。

2.4 位置编码：为无序的计算注入顺序

RNN 天然知道顺序——按序输入。Transformer 并行喂入所有 token，序列顺序丢失了。位置编码就是给每个 token 打上"我是第 i 个"的标签。

原始论文用正弦/余弦函数编码位置：

```
PE(pos, 2i) = sin(pos / 10000^(2i/d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i/d_model))
```

这个方案有个优雅的性质：PE_{pos+k} 可以由 PE_pos 的线性变换得到——意味着模型可以学到相对位置关系。但实际效果一般。

学习式位置编码（BERT 用的）是另一个方向：直接用可训练的 embedding 查表，简单有效，但无法外推到训练时未见过的长度。

真正改变格局的是 **RoPE**（Rotary Position Embedding，Su et al., 2021）：不对输入向量加位置编码，而是对注意力计算的 Q 和 K 向量做**旋转**——旋转角度由位置决定。这样 Q^T K 的点积值天然包含了相对位置信息（因为旋转矩阵的性质：Rot(m)^T Rot(n) = Rot(n−m)）。LLaMA、Mistral、Qwen、DeepSeek 全都采用了 RoPE。

ALiBi（Press et al., 2021）更激进：直接去掉位置编码，在注意力分数上加一个线性偏置，越远的 token 偏置越大。简单到令人发指，但 BLOOM 和 MPT 证明了它能外推到训练长度 10 倍的序列。

2.5 FFN 层：被低估的记忆仓库

每层 Transformer 有两个子层：Multi-Head Attention 和 Feed-Forward Network。大家在聊 Attention，很少聊 FFN。

但 FFN 占了 Transformer 总参数的三分之二。Geva et al.（2021）给出过一个漂亮的观点：**FFN 是一个 key-value 记忆库**。第一层线性变换(W_1)的输出是"检测到的模式"，激活函数（ReLU/GELU/SwiGLU）是阈值，第二层线性变换(W_2)是"对应的输出"。

所以 Attention 负责"从上下文检索相关信息"，FFN 负责"从参数记忆中提取知识"。Attention 回答"这段文本里说了什么？"，FFN 回答"根据你的训练知识，接下来该是什么？"。

MoE 的切入点恰恰在这里。既然 FFN 是一个记忆库，为什么每个 token 都要查询整个记忆库？能不能让 token 只查询最相关的几个"记忆分区"？这就是 MoE-Transformer 的核心动机：**把 FFN 拆成多个 expert，每个 token 通过门控网络只激活 1-2 个 expert**。计算量不变（激活参数不变），但总参数量可以大幅增加——模型"知道"的更多，"花的钱"一样。

2.6 为什么 Transformer 能 scale

Scaling 的前提是三个能力：

1. **完全并行**：训练时每个 token 同时看到所有上下文，GPU 全部核心都在干活。对比 RNN 的串行瓶颈，这是代际差异。

2. **恒定路径长度**：任意两个 token 之间的信息路径都是 O(1)——一层 attention 就能直接传递。RNN 要走过整个序列长度。这是长距离依赖能力的根因。
3. **结构极度规整**：所有层结构相同、所有位置处理方式相同。想加参数？加层、加宽、加头——没有架构瓶颈挡路。RNN 同样可以堆叠，但梯度消失限制了深度；Transformer 的残差连接 + LayerNorm 让几百层的网络也能稳定训练。

这三个能力叠加，再加上 Scaling Laws 的发现——模型性能和算力、数据、参数量呈平滑幂律关系——让 Transformer 成了第一个真正"投入更多算力就能稳定变强"的架构。这是它统治力的根源。

2.7 训练与推理：Transformer 的双面人生

很多人学了 Transformer 的完整架构，但搞不清**训练和推理时发生了什么本质不同**。这是面试最常踩的坑。

训练时（Teacher Forcing）：

所有 token 同时进入模型，一次前向计算出所有位置的预测。因为 Ground Truth 已知（"the cat sat"），模型可以并行计算：

- 输入："[BOS] the cat sat"
- 目标："the cat sat [EOS]"
- 一次前向，对所有 token 位置求交叉熵 loss
- 用 Masked Self-Attention 防止偷看未来 token（attention 矩阵上三角置 −∞）

推理时（自回归生成）：

模型只有一个初始 token "[BOS]"，然后进入循环：

```
t=1: 输入 [BOS] → 输出概率分布 → 采样 "the"
t=2: 输入 [BOS, the] → 输出概率分布 → 采样 "cat"
t=3: 输入 [BOS, the, cat] → 输出概率分布 → 采样 "sat"
t=4: 输入 [BOS, the, cat, sat] → 输出概率分布 → 采样 [EOS] → 停止
```

每一步都要重新前向——Transformer 推理的 $O(n^2 \cdot L)$ 复杂度让长序列推理很贵。

KV Cache：推理加速的关键技巧

推理时，每次新 token 进来都要重新计算之前所有 token 的 K 和 V。这纯属浪费——"the"的 Key 和 Value 在上一步已经算过了，算完就没再用。

KV Cache 的做法：把每一步算出的 K、V 保存在显存里，下一步计算时直接用缓存的值，只算新 token 的。显存占用降为 O(n)（只存不重复算），但 KV cache 本身也要吃显存——长序列的 KV cache 可能比模型参数还大。

这也是为什么 GQA、MQA、MLA 这些注意力压缩方案如此重要——它们的核心目标都是**缩减 KV cache 大小**，而不是缩减计算量。做推理优化的工程师眼里，Transformer 的瓶颈顺序是：显存带宽 > 计算吞吐 > 模型大小。

2.8 分词：Transformer 的「识字」方式

Transformer 处理的不是单词、不是汉字，是 **token**。Tokenizer 把原始文本切成 token 序列。

BPE（Byte Pair Encoding） 是主流方案：从单个字符开始，统计出现频率最高的相邻字符对，合并成一个新 token。反复执行直到词表满。结果是高频词自成一个 token（"the" → 1 个 token），低频词拆成子词（"transformer" → "trans" + "form" + "er"）。

中文分词一般用字符级 BPE 或 SentencePiece。一个汉字通常是 1–2 个 token，一个英文单词也是 1–2 个 token。

分词为什么重要？ 直接影响模型能力和成本——"pheromone"如果被分词模型切成 phe + rom + one，模型就无法感知"信息素"这个概念。tokenizer 的词表大小也直接影响 embedding 层参数量。而且不同模型的 tokenizer 不通用，切同一个句子结果不同。这是做模型对比时最容易忽略的维度。

2.9 前置知识速查：一张图谱让你串起来

```
文本输入
  ↓ Tokenizer → 分词成 token 序列
  ↓ Embedding + Positional Encoding → 向量表示
  ↓
  ↓ [开始 1 层 Transformer]
  ↓   ├─ Multi-Head Self-Attention（QKV → 加权信息聚合）
  ↓   │    └─ 跟 MoE Router 同源：「软路由」哲学
  ↓   ├─ Add & LayerNorm（残差连接 + 归一化）
  ↓   ├─ FFN（Key-Value 记忆库 → 提取知识）
  ↓   │    └─ MoE 替代点：把一个大 FFN 拆成多个 expert
  ↓   └─ Add & LayerNorm
  ↓ [重复 N 层]
  ↓
  ↓ 输出层 → 映射到词表 → softmax → 采样下一个 token
```

这也解释了为什么 MoE 分析是理解 Transformer 的天然参照：**两者共享"软路由 + 记忆检索"的核心隐喻**，只是在不同层级实现——Attention 在信息层做路由，MoE 在计算层做路由。

三、纵向分析：从一场豪赌到统治世界

3.1 诞生（2017）：论文名叫"Attention Is All You Need"的时候，没人信

2017 年 6 月 12 日，Google Brain 的八位作者提交了论文到 arXiv。八位作者是 Vaswani、Shazeer、Parmar、Uszkoreit、Jones、Gomez、Kaiser、Polosukhin。论文名致敬披头士的 "All You Need Is Love"。

当时的环境是这样的：在机器翻译上，大家公认 attention 有用（Bahdanau et al., 2014），但所有人都想的是"在 RNN 上加 attention"。Jakob Uszkoreit 2016 年在 Decomposable Attention 实验里怀疑 attention 本身可能就够了——这个想法在 Google Brain 内部一度被当作"你是不是疯了"之类的异端。

"去掉 RNN，只用 attention"——这个决策的勇气，回过头看，比技术本身更关键。它意味着放弃 NLP 领域三十年来关于"序列建模必须有递归结构"的默认假设。于是他们整出了位置编码这种完全人造的、强行注入顺序信息的方案来弥补。

初始架构放到今天看小得可笑：6 层 encoder + 6 层 decoder，d_model=512，8 个 attention head，FFN 维度 2048，总共约 6500 万参数。在 8 块 P100 GPU 上训练 3.5 天，WMT 2014 英德翻译 BLEU 28.4——新 SOTA，但不是什么惊天动地的数字。

论文在 NeurIPS 2017 没拿最佳论文奖，甚至没拿到 Oral 展示，只是 Poster。如今引用量已破 17 万次，极有可能是 21 世纪被引最多的论文。

讽刺吧？那届 NeurIPS 没人意识到它有多重要。

3.2 分歧元年（2018）：BERT 和 GPT 分道扬镳

2018 年 6 月，OpenAI 发了 GPT-1——117M 参数的 decoder-only Transformer，自回归语言模型。

2018 年 10 月，Google 发了 BERT——340M 参数的 encoder-only Transformer，Masked Language Model 双向预训练。

同一年、同一个基础架构、两个完全相反的设计选择。

维度	GPT-1	BERT
时间	2018.06	2018.10
参数	117M	340M
架构	Decoder-only	Encoder-only
预训练	自回归 (next token)	MLM (masked token)
上下文	单向	双向
擅长	文本生成	文本理解
背后公司	OpenAI	Google

为什么选不同方向？因为目标不同。

BERT 瞄准的是"理解"——分类、问答、命名实体识别。双向上下文是理解任务的最优解，遮一个词让模型从左右两个方向猜，这天然是 encoder 干的事。

GPT 从一开始就以"生成"为目标。自回归生成天然是 decoder 的事。而且 decoder-only 推理时只有一个模型在跑，不用 encoder 先编码一遍——架构更简洁，推理更快。

这个分歧定义了此后五年的 NLP 版图。BERT 统治了理解任务（GLUE 榜单从 2018 到 2020 被 BERT 系刷屏），GPT 在生成任务里走了更远的路。最终，decoder-only 赢了——不是因为理解不重要，而是因为生成是更底层的元能力：你只要能生成，理解自然隐含在其中。

3.3 规模爆发（2019-2020）：GPT-2 不让开源，GPT-3 震了所有人

2019 年 2 月，GPT-2 发布。1.5B 参数，分阶段放出——OpenAI 说太危险，怕被滥用生成假新闻。当时被学术界群嘲为"炒作"，但现在回头看，那是 AI 安全讨论进入主流视野的起点。

GPT-2 真正的技术价值不在规模，在**零样本能力**。不加任何 fine-tune，直接让它续写，就能完成翻译、摘要、问答。In-context learning 的种子埋下了。

2019 年的架构探索不只是 GPT 和 BERT 两条线。RoBERTa 做了"BERT 但更多"——去掉 NSP（这是 BERT 里最弱的贡献）、更大的 batch、更多数据、更久训练，性能稳定超越 BERT。T5 走了 Encoder-Decoder 路，用 "Text-to-Text" 统一框架把所有 NLP 任务变成一个格式。XLNet 用排列语言建模试图结合双向和自回归，虽然性能好但训练太贵，没流行起来。Transformer-XL 引入片段级递归和相对位置编码解决长文本问题，后来被 XLNet、PaLM 吸收。

2020 年两篇论文彻底改变了走向。

Scaling Laws（Kaplan et al., 2020, OpenAI）：模型性能随参数、数据、算力三者遵循平滑幂律关系。关键结论是"增加参数比增加数据更有效"——73% 的额外算力应该分配给扩大模型参数。这个公式直接导致了 GPT-3（175B 参数，但只用了 ~300B tokens 训练）和 Gopher（280B 参数，300B tokens）。

GPT-3（2020 年 5 月）：175B 参数，纯 decoder-only Transformer。技术架构上没有根本创新——就是把 GPT-2 放大了 100 倍。但发现了 in-context learning——推理时不 fine-tune，只给 few-shot 示例就能完成任务。这个发现比任何架构创新都重要：它暗示了"算力 + 数据 + 注意力"这个配方，能涌现出我们没有明确定义的智能形态。

3.4 架构收敛（2021-2022）：RoPE 上位，MoE 入侵，Scaling Laws 被颠覆

2021 年是位置编码的一次宫廷政变。RoPE（Su et al., 2021）用复数旋转矩阵为 Q/K 注入相对位置信息，数学优雅、实现简单、外推性好。ALiBi（Press et al., 2021）更激进——直接去掉位置编码，加个线性偏置完事。两者都解决了"训练短、推理长"的矛盾。

最终 RoPE 胜出，成了几乎所有大模型的标准配置。回头看不难理解——RoPE 在理论优雅性（旋转群的性质直接编码了相对距离衰减）和实际效果（LLaMA 系列验证了上下文外推能力）之间撞到了最优点。

同一年，Google 在做一个疯狂的路线上探索。GShard（2020/2021）第一次把 MoE 大规模应用到 Transformer：每隔一个 FFN 层换成 MoE 层，top-2 路由，总参数飙到 600B。Switch Transformer（Fedus et al., 2021）更进一步——top-1 路由，最简单的方案，做到了 1.6 万亿参数（2048 个 expert），而且速度比 T5-XXL 快 4 倍。

这个路线的意义到今天才真正被广泛认知：**MoE 解耦了"模型知道多少"和"每次推理花多少钱"**。总参数量像硬盘，激活参数量像 RAM。硬盘可以很大，每次只读需要的扇区。

2022 年，DeepMind 的 **Chinchilla** 论文（Hoffmann et al., 2022）颠覆了 Kaplan 的 Scaling Laws。

Kaplan 说 73% 算力投给参数、27% 投给数据。Chinchilla 说各 50%——最优 tokens/参数比是 ~20，而不是 ~1-2。GPT-3 用了 ~1.7 tokens/参数，按 Chinchilla 的标准严重"欠训练"。

为什么 Kaplan 错了？方法问题：固定学习率调度，没有充分探索 tokens/参数比空间，在浅层数据做了外推。这是一个典型的"实验设计不够好→结论偏差→行业走偏"的案例。

Chinchilla 70B 用 1.4 万亿 tokens 训练，以 1/4 的参数量超越了 GPT-3 175B。它开启了"小模型 + 海量数据"的新路线，直接催生了 LLaMA 系列。

同年，**PaLM**（540B，Google）登场：密集 Decoder-only，118 层，d_model=18432，48 个 head。三件事值得一提：SwiGLU 激活函数成为新标准（ReLU → GELU → SwiGLU）；Parallel Layers（注意力和 MLP 并行计算，不串行），训练速度提升 15%；RoPE + Multi-Query Attention 的组合。PaLM 的单头 KV 共享设计虽然牺牲了注意力表达能力，但推理时 KV cache 降低了若干倍——这个取舍在 2024 年 DeepSeek-V2 的 MLA 之前，一直是大模型推理优化的主线。

3.5 开源反击（2023）：LLaMA 绝杀，FlashAttention 把 $O(n^2)$ 打下来

2023 年是"小团队也能训大模型"的一年。

LLaMA（Touvron et al., Meta, 2023 年 2 月）是转折点。严格遵循 Chinchilla 公式——LLaMA 65B 用了约 22 tokens/参数，1.4T tokens——架构选择出奇保守：RoPE + SwiGLU + RMSNorm + Pre-Norm。没有花活。结果呢？LLaMA 13B 在多数 benchmark 上超越 GPT-3 175B。

这件事传递了一个信息：**架构不需要什么重大创新，把 Scaling Laws 吃透、把数据质量搞好，就能用小得多的模型达到同等性能**。更重要的是，LLaMA 开源了权重（虽然经过了那场著名的泄露争议），引爆了整个开源模型生态。LLaMA-Adapter、Alpaca、Vicuna——开源社区第一次能在大模型上跟闭源叫板。

GPT-4 也在这年 3 月发布。架构细节 OpenAI 没公开——这是他们第一次不透明。但多方分析（SemiAnalysis 等）推测：~1.8T 总参数，MoE 架构（16 个 expert，每 token 激活 2 个），激活参数约 280B。这意味着 GPT-4 已经是 MoE-Transformer 了。

与 GPT-4 几乎同时，**FlashAttention**（Dao et al., 2022/2023）获得了 NeurIPS 最佳论文提名。核心创意：注意力瓶颈不在计算而在内存带宽。Q、K、V 矩阵在 HBM 和 SRAM 之间反复搬运，GPU 大量周期空转。FlashAttention 用 tiling 技术把矩阵分块留在 SRAM 里算完再搬——2-4x 加速，O(N) 内存消耗。

这是一个"工程创新超越算法创新"的经典案例：算法完全等价（都是精确注意力），只是重新排了一下计算顺序。

3.6 MoE 大爆发与 DeepSeek 的奇迹（2024）

2024 年是 MoE-Transformer 真正成为主流的年份。所有人都在做。

Mixtral 8x7B（Mistral, 2023 年底）：46.7B 总参数 / 12.9B 激活，8 个 expert 里每次选 top-2。性能匹配 LLaMA 2 70B，推理速度快 6 倍。社区疯狂就是因为这个——12.9B 参数的推理成本，换来 70B 级别的能力。

DBRX（Databricks, 2024）：132B / 36B 激活，top-4 / 16 expert。企业级定位，但社区热度不如 Mixtral。

Gemini 1.5 Pro（Google, 2024.02）：MoE + 1M context window。MoE 设计推测是 top-2 / ~32 expert。用 Ring Attention + 上下文并行实现了百万级上下文。

Phi-3 和 **Gemma 2** 这些"小而美"路线也值得关注。两者都是密集架构（非 MoE），用蒸馏 + 精选数据在 3-14B 参数区间做到了惊人的性能。

但 2024 年真正的主角是 **DeepSeek**。

DeepSeekMoE（2024.01）提出了细粒度专家分割——把 FFN 中间维度拆成更小的 expert，让 token 激活更多 expert 以达到灵活组合，同时设"共享专家"来捕获通用知识。这个方案把 MoE 从"少而大的专家"推向"多而小的专家"。

DeepSeek-V2（2024.05）放了大招：**Multi-Head Latent Attention（MLA）**。核心思想是，多头注意力各头之间的 K、V 有大量冗余，把 K/V 压缩到低维潜空间（矩阵分解），解耦了"计算精度"和"存储成本"——KV cache 降 93.3%，训练成本降 42.5%，吞吐量提升 5.76 倍。这个创新直接让 KV cache 的瓶颈几乎消失。

DeepSeek-V3（2024.12，技术报告 12 月公开）：671B 总参数 / **37B 激活**（仅 5.5%）——1 个共享专家 + 256 个路由专家，每 token 激活 9 个专家。但这还不是最吓人的部分：

- **无辅助损失负载均衡**：MoE 训练传统上用辅助损失强制 token 均匀分配到大专家，操作麻烦且会影响性能。DeepSeek V3 用可学习的 per-expert bias 动态调节，干脆去掉了辅助损失。

- **Multi-Token Prediction (MTP)**：每步预测 2 个 token，不是只预测下一个。第二个 token 接受率 85-90%，能显著加速推理（speculative decoding）。
- **FP8 混合精度训练**：业界首个大规模 FP8 训练，用块级缩放解决数值不稳定。结合 DualPipe 算法（前向/反向计算与 MoE all-to-all 通信完全重叠），MFU（算力利用率）达到惊人的 ~70%——GPT-4 大约 32-36%。
- 训练成本仅 557 万美元。同等性能的 LLaMA 3 405B 估计花了 5.5 亿美元。十分之一。

"低成本实现最先进性能"——DeepSeek-V3 证明了一件事：**MoE + 工程极致优化 > 单纯堆算力**。而且不需要顶级的 GPU（用的是 H800，性能受限版本）。

3.7 2025-2026 趋势总结：替代架构不再是"总有一天"

截至 2026 年 4 月，各旗舰模型的架构选择清晰地展示了行业共识：

模型	架构	注意力	激活参数/总参数
DeepSeek-V3	MoE	MLA	37B / 671B (5.5%)
GPT-4	MoE	MQA	~280B / ~1.8T (15.6%)
Gemini 2.5 Pro	MoE	MHA (推测)	~50B / ~900B (5.6%)
LLaMA 4	MoE	GQA	~17B / ~400B (4.3%)
Mistral Large 3	MoE	GQA/MLA	~50B / 675B (7.4%)
Qwen3 (MoE)	MoE	GQA	~22B / 235B (9.4%)
Claude 4 Opus	**Dense**	MHA	**100%**
Qwen3.5	混合	DeltaNet+Attention	—

Anthropic 的 Claude 4/4.5 系列是唯一坚持 **Dense 架构**的旗舰模型。这个选择不是技术落后——是设计哲学：所有参数在每个 token 上都激活，每个 token 都经过了完整的模型推理。Anthropic 显然认为这种"整体性"对于深度推理和连贯性是值得的代价。

替代架构这边，Mamba（Gu & Dao, 2023）把选择性状态空间模型从信号处理带入 NLP——O(n) 复杂度，6x 吞吐，1/3 内存，2024 年底的 Mamba-2 进一步提升了效率。RWKV 走了一条"训练像 Transformer、推理像 RNN"的混合路线。各种 Linear Attention、Gated DeltaNet 的尝试也在推进。

2026 年最值得关注的信号是 **Qwen3.5**——用 Gated DeltaNet 替换了 75% 的标准 attention 层。不是全部替换，是混着用。这个务实的方向很可能成为这个十年的答案：**不是 Transformer vs X，是 Transformer + X 的最优配比**。

四、横向分析：九年后，Transformer 站在什么位置

4.1 已经没有"单一 Transformer"了——六大架构派系

九年时间下来，"Transformer"这个词已经不足以描述一个具体的技术栈。今天的 Transformer 是一个树状分支的演化树：

Dense Decoder-Only（GPT-2/3, LLaMA 1/2/3, Claude 4, Qwen3-Dense）

：主流通用方案。纯自回归，每个 token 经过全部参数。Claude 4 是唯一还在坚持这条路的旗舰闭源模型。优势在一致性和深度推理，劣势在推理成本。

Sparse MoE Decoder-Only（Mixtral, DeepSeek V2/V3, Gemini, DBRX, Qwen3-MoE）

：2025-2026 大模型的主流。参数量大但激活量少，总参数是"硬盘"，激活参数是"RAM"。当前所有 100B+ 参数的新模型（除 Claude 外）都用了 MoE。

Encoder-Only（BERT, RoBERTa, DeBERTa, ModernBERT）

：理解任务的专家。预训练只关心双向上下文表示。如今主要活跃在检索、分类等场景。2024 年底的 ModernBERT 给这个路线加了一针强心剂——用 RoPE、FlashAttention、更高效的 tokenizer 把 BERT 架构现代化，在处理长文档上重新获得了竞争力。

Encoder-Decoder（T5, BART, UL2, Flan-T5）

：翻译和摘要的天然合适架构，但被 Decoder-only 替代的趋势不可逆。Flan-T5 可能是这个派系最后的代表作。

非 Transformer（Mamba, Mamba-2, RWKV, Hyena, StripedHyena）

：都在承诺 O(n) 或 O(n log n) 复杂度。2025-2026 年在小规模和中等规模验证了有效性（1-7B 参数区间确实有竞争力），但在 70B+ 级别还没证明能完全取代 Transformer 的 scaling 能力。

混合架构（Jamba, Zamba, Qwen3.5, Samba）

：2025 年起爆发。核心思路：长序列让 SSM/线性注意力处理（省计算），推理让标准注意力处理（保精度）。Qwen3.5 用 75% Gated DeltaNet + 25% 标准注意力是最激进的实践。这个方向最像"正确答案"——取各自所长，而不是誓死效忠某一种。

4.2 注意力机制：MLA vs GQA vs MHA 的三方会战

注意力变体的竞争是最近两年最大的架构故事：

方案	核心做法	KV Cache	代表模型
MHA	每个头独立 K,V 投影	最大	Claude 4, Gemini
MQA	所有头共享 K,V	1/h	GPT-4 (推测)
GQA	分组共享 K,V (如 8 头分 2 组)	1/g	LLaMA 3, Qwen3, Mistral
MLA	K,V 低秩压缩到潜空间	1/10x	DeepSeek V2/V3, Mistral Large 3 (跟进)

MLA 是 2024 年最值得关注的注意力创新。DeepSeek 将 KV 投影矩阵做低秩分解（W_K ≈ W_K_d × W_K_u），信息空间远小于参数空间——因为多头之间的 K、V 存在大量重叠信息。KV cache 降一个数量级，命中率和 GQA 相当甚至更好。

MLA 向行业发出的信号很明确：**GQA 不是注意力压缩的尽头**。低秩压缩利用了 heads 之间的结构冗余，而 GQA 只是简单地把 head 分组——前者利用信息论原理，后者只是工程折中。

4.3 MoE 设计的四大分歧

即使都是 MoE-Transformer，设计路线也有根本差异：

专家数量：少而大（LLaMA 4: 2 个活跃专家）vs 多而小（DeepSeek V3: 256 个专家，激活 9 个）。DeepSeek 认为更多的小专家更容易专业化、更适合组合表达复杂模式。LLaMA 4 认为少量大专家训练更稳定、推理更简单。

路由策略：Top-2（传统，Mixtral、Gemini）vs Top-1（Switch Transformer）vs 设备限制路由（DeepSeek 用设备限制辅助损失控制跨节点通信）vs aux-loss-free（DeepSeek V3）。负载均衡一直是最核心的工程难题。

共享专家：有（DeepSeek、GShard）vs 无（LLaMA 4、Qwen3 MoE）。DeepSeek 认为有一个始终激活的共享专家可以捕获通用知识（语法、常识），让路由专家专注特定领域，减少参数冗余。这个选择目前看更合理——通用模式不应该被路由选择割裂。

细粒度：1x FFN = 1 expert（传统）vs 1x FFN = 多个小 expert 组合（DeepSeek）。细粒度支持更灵活的知识组合，但工程实现和负载均衡更难。

4.4 Claude 为什么还在坚持 Dense

这是 2026 年 Transformer 版图上最有意思的问题。

在所有人都用 MoE 的 2025-2026 年，Anthropic 的 Claude 4/4.5 仍然选择 Dense 架构。可能的原因：

1. **整体推理**：Dense 架构中每个 token 经过全部参数，没有"不同 token 走不同子网络"的不可控性。对于需要深度思考、长链推理的任务，这种一致性可能很重要。MoE 的专家切换可能引入微妙的推理断裂——目前没有直接证据，但这个担心在 Dense 阵营中广泛存在。
1. **创意与模糊任务**：MoE 的优势是专业化，但在高度模糊的、需要跨领域综合的创意任务中，"所有人都参与讨论"可能比"少数专家做决定"更有效。
1. **Scaling 的考量**：Anthropic 可能认为，在它们的目标规模上，Dense 的 scaling 曲线还没有进入边际递减区域。MoE 的优势在数千亿参数以上才真正显现，如果 Anthropic 的模型规模没有达到需要 MoE 的程度，Dense 是更简单的选择。
1. **训练稳定性**：MoE 训练更难、更不稳定，DeepSeek 花了巨大工程努力才驯服它。对于一家以安全为核心考量的公司，训练稳定性可能是更重要的指标。

这本质上是两种智能理念的碰撞：**"专业化分工"vs"整体协商"**。结构化的推理任务（数学、编程）更适合专业化，需要跨领域直觉综合的创意任务更适合整体协商。未来可能不是二选一，而是动态选择——根据任务类型调整激活模式。

4.5 三种 Scaling 路径的殊途

2026 年，扩大模型能力不再只有"多训几层"这一条路。三条路径在同时推进：

预训练 scaling（更多参数 + 更多数据）

：进入边际递减。高质量预训练数据几近耗尽。模型越来越大的收益越来越小——GDP 级别不能再靠 10x GPT 实现了。但小模型仍然有显著收益。

推理时计算 scaling（Chain-of-Thought + Extended Thinking）

：2025-2026 年增长最快的能力维度。OpenAI o1/o3、Claude 4 Extended Thinking、Gemini Deep Think 都证明了一点：让模型多看几秒，比多训几个月更划算。DeepSeek-R1 用 GRPO RL 强化了推理能力，纯粹靠 RL 而不是 supervised fine-tuning。这意味着推理能力的规模化路径可能和预训练完全不同。

后训练 scaling（RL/DPO/efficiency curation）

：精细数据配比 + RLHF/DPO + Constitutional AI + 合成数据。这块 DeepSeek 的 GRPO 和 OpenAI 的 deliberative alignment 是两个代表。在预训练 scaling 减速的背景下，后训练的重要性越来越高。

三条路径的关系不是"选一个"，而是"找最优比例"。就像 2022 年 Chinchilla 给出了预训练中参数和数据的比例公式，下一个突破口可能是"预训练 vs 推理计算 vs 后训练"的三元 optimal allocation law。

五、横纵交汇洞察

5.1 历史的必然与偶然

回头看 Transformer 九年的发展史，有几个节点让人意识到——历史不只是被大趋势推动，也由具体的决策和质量把控决定。

2017 年 Vaswani 团队的赌博。"去掉 RNN"这个选择，按当时的常识是鲁莽的。但这个鲁莽决策一旦证明可行，就打开了 RNN 从未触及的空间——模型性能随算力平滑增长。RNN 的架构本身限制了 scaling，Transformer 没有这个上限。这个决策在当时被低估了，今天看可能是 AI 历史上最重要的架构决策。

2022 年 Chinchilla 纠正 Scaling Laws。如果 Chinchilla 没有发，整个 2023-2024 年的方向可能还是"超大模型 + 欠训练"（GPT-3 和 Gopher 的路线）——用更烧钱的方案取得更差的效果。LLaMA 可能不会诞生，开源模型生态可能要晚一年。一篇论文挽救了多少浪费的 GPU 小时。

2023 年 LLaMA 开源。尽管泄露过程有争议，LLaMA 的开放权重直接催生了整个开源大模型生态。这是偶然大于必然的一个节点——如果那批权重没泄露、如果 Meta 没有顺势开源，2024 年的 AI 格局会完全不同：没有 Mistral、没有 Qwen 的开源版、没有 DeepSeek 从开源序列中学到的经验。LLaMA 的开源是 2020 年代影响力最大的技术发布之一。

2024 年 DeepSeek 连发三篇。DeepSeekMoE → DeepSeek-V2 (MLA) → DeepSeek-V3 (无辅助损失 + FP8 + MTP)，三篇论文把 MoE 从"可行"推到"极致"。MLA 很可能成为注意力压缩的新标准（已有 Mistral Large 3 跟进）。更重要的隐含信息是——**557 万美元训练一个 SOTA 模型**——这个数字是对硅谷"用美元堆出来的壁垒"的一次祛魅。

5.2 从 MoE 的视角重新审视 Transformer

回到最初的问题意识。MoE 给了一个理解 Transformer 的特别视角。

Transformer 的注意力本身就是一个"软路由"：每个 token 通过 attention 动态路由到其他 token 的信息。FFN 是"记忆库"。MoE 把"软路由"的哲学推进到 FFN 层——不只是"路由信息"（attention），也"路由计算"（MoE FFN）。

从这个角度看，Transformer 的演进方向是一贯的：**让每个 token 更精准地获得它"需要的信息"和"需要的计算"**。Dense Transformer 是第一代——所有计算、所有信息都给所有 token。MoE-Transformer 是第二代的早期形态——

计算有选择性了，信息还是全给。MLA 是另一个维度——减少存储冗余，而不减少精度。未来是什么？可能是"更端到端的软路由"：不只在注意力层和 FFN 层各自路由，而且跨层、跨时间步地动态路由。

SSM 和 Linear Attention 从另一个方向切进来——如果 $O(n^2)$ 的注意力在长序列上太贵，能"把序列做预压缩再喂给 attention attention 只聚焦关键片段"吗？Qwen3.5 的混合方案是最接近的雏形。

5.3 三个未来剧本

最可能剧本（2026-2028）：Dense-MoE 双轨并行的温和演进

Transformer 继续主导，但"Dense vs MoE"变成"任务依赖的动态架构"。短序列、生成密集型任务用 Dense（降工程复杂度），高吞吐推理用 MoE。注意力以 GQA 为主，MLA 逐步渗透。混合架构（SSM/线性模型 + 稀疏注意力）从当前 10-75% 的混合比例找到最优配比。推理时计算成为和预训练同等重要的能力维度。

Dense 不死。就像汽车没有淘汰自行车——当你的需求不是"最高效"，而是"最均衡"，Dense 的优势依然成立。但 MoE 会吃掉越来越多的份额。

最危险剧本：Transformer 的 scaling wall 在 2028 年前到来

预训练数据接近物理上限（人类文本总量就那么多），模型继续放大带来的边际收益趋近于零。所有模型性能趋同——因为都是用同一批数据、同类架构训练的。差异化只来自后训练（RLHF、GRPO、curriculum curation）和推理时计算。规模竞争变成"谁更有创意地使用有限数据"。

这种背景下，架构不再是护城河。真正区别在数据飞轮——谁能更快收集真实用户交互数据、谁能生成更高质量的合成数据。

最乐观剧本：新注意力范式出现，$O(n^2)$ 被 $O(n \log n)$ 攻破

Mamba / RWKV / Gated DeltaNet / Linear Attention 的某个继任者，在 70B+ 参数规模上证明了与 Transformer 的对等性能，同时保持 10x 的吞吐优势。Transformer 的位置编码、FlashAttention、MLA 这些优化可以被替代架构继承——因为它们的改进主要在计算层面，不在范式层面。

最关键的是 training-inference efficiency parity——替代架构需要不仅在推理时快、在训练时也高效（因为训练是 scaling 的核心瓶颈）。Mamba 在这方面走的最远（FlashAttention 的作者 Tri Dao 同时也是 Mamba 的合著者不是巧合）。

我觉得最可能的混合场景是"最可能"和"最乐观"并行：混合架构的比例越来越高，0-5 年间 SSM/线性层可能从 25%（Qwen3.5）逐步升到 50-75%。纯 Transformer 在短期仍主导，但在效率关键场景的份额在缩小。真正的范式切换在 5 年以后。

5.4 九年前的披头士

"All You Need Is Love"——"All You Need Is Attention"。

2017 年的八位作者大概率没意识到他们在写 AI 史上最重要的论文之一。一篇 NeurIPS Poster，在九年后拥有十七万引用。这条纵向线的最后一个回环：**最好的研究往往不是当时看起来最耀眼的，而是最难被推翻的**。Transformer 没有被推翻——它在演化、在被拼接、在被挑战——但"用注意力替代所有序列建模"这个核心洞察，历经 BERT/GPT 的分裂、Scaling Laws 的起落、MoE/MLA 的入侵、SSM 的挑战，至今没有被动摇。

下一个九年会不会有答案？值得等。

六、实战：面试题、场景题、应用题

面试题

Q1: Self-Attention 的计算流程是怎样的？Q、K、V 分别是什么角色？

这是 Transformer 面试必问题。

计算步骤：

1. 输入 X（shape: seq_len × d_model）
2. 线性投影：Q = X·W_Q, K = X·W_K, V = X·W_V
3. 计算注意力分数：Scores = Q·K^T / √d_k
4. 因果 mask（decoder）：上三角置 −∞
5. Softmax 归一化：Attention Weights = softmax(Scores)
6. 加权输出：Output = Attention Weights · V

QKV 的角色：Query 是"我要找什么"，Key 是"我能提供什么"，Value 是"我的实质内容是什么"。这和数据库检索（Query→Index→Content）高度同构。

常见追问：为什么要除 √d_k？因为点积方差随 d_k 线性增长，大值进入 softmax 会导致梯度几乎为零。除以 √d_k 把方差控制回 1。

Q2: Multi-Head Attention 为什么比单头好？头数越多越好吗？

每个头在不同的低维子空间计算注意力，学到不同类型的依赖关系。有的头关注相邻句法结构，有的头关注长距离语义。多个头让一个 token 在一步内同时形成多种关系模式。

头数不是越多越好。头数增加 → 每个头的维度减小 → 表达能力可能不足。每个头的维度 d_k 至少要能容纳有意义的表示（通常 64 起步）。LLaMA 70B 用了 64 个头 / d_k=128，这是一个经过验证的平衡点。现代大模型更多用 32-64 个头配上 GQA/MQA 来减少冗余。

Q3: 为什么要用 Positional Encoding？RoPE 为什么优于 Sinusoidal？

Transformer 并行喂入所有 token，没有序列顺序的概念——"我爱北京天安门"和"门安天京北爱我"对这层矩阵来说没区别。

Sinusoidal（原始方案）：外推性差，训练长度之外的序列位置分布偏移。

RoPE 赢在两点：一是用**旋转矩阵**的数学性质优雅地编码相对距离（旋转 m 度再旋转 n 度 = 旋转 n−m 度），二是**外推性**——虽然也不完美，但对训练分布外的长度泛化比 Sinusoidal 好得多。

另一个值得提的方案是 **NoPE**（2024 年的探索）：有实验表明，在足够长和大规模的训练下，Transformer 可以学到隐式位置信息。但目前在收敛速度和效果上仍不如 RoPE。

Q4: LayerNorm 在 Transformer 中为什么用 Pre-LN（前置）？

原始论文用 Post-LN（Attention 后面接 LN），后来大家发现 Pre-LN（LN 放在 Attention 前面）训练更稳定。

原因：Post-LN 中，残差分支的梯度要穿过 LayerNorm 才能回到输入——这个路径更复杂、容易梯度爆炸。Pre-LN 中残差分支有干净的恒等映射路径，梯度直接流过。这在深层网络中（>20 层）区别巨大。

现代所有大模型（LLaMA、GPT-4、Claude、DeepSeek）都用的 Pre-RMSNorm（LayerNorm 的简化版，去掉了均值的 centering，只做 scaling）。

Q5: 为什么 Transformer 能 scale 到万亿参数，而 RNN/LSTM 不行?

三个原因：

1. **并行性**：RNN 每一步依赖前一步结果，1024 个 token 就要串行 1024 步。Transformer 的注意力是矩阵乘法，1024 个 token 一次并行算完。
2. **路径长度**：Transformer 中任意两个 token 信息传输只需 O(1) 层。RNN 要走 O(seq_len) 步，梯度衰减严重。
3. **正则化**：残差连接 + LayerNorm 让几百层 Transformer 稳定训练。堆叠 100 层 LSTM 梯度早消失了。

但最关键的是 **Scaling Laws** 的幂律关系——投入更多算力，Transformer 性能稳定线性提升。RNN 找不到这个规律，加算力不一定更好。

Q6: Transformers 的 $O(n^2)$ 复杂度问题怎么解决?

按路线分四类：

- **稀疏注意力**（Longformer、BigBird）：不是所有 token 对都算注意力，只有局部窗口 + 几个全局 token
- **线性注意力**（Linformer、Performer）：用核函数重新排列矩阵乘法顺序，O(n) 替代 $O(n^2)$
- **IO 优化**（FlashAttention）：算法不变，重排计算顺序减少 HBM↔SRAM 搬运
- **替代架构**（Mamba、RWKV）：完全换掉 attention，用状态空间或线性递归

工业界目前的实用路线是 FlashAttention + GQA + 上下文分块，学术界的长期目标是替代架构。

Q7: MoE 是什么？跟 Dense Transformer 比有什么优劣?

MoE（Mixture of Experts）把 FFN 拆成多个 expert，每个 token 只激活 1-2 个 expert，不激活所有参数。

优势：相同推理成本下，模型可以"知道"更多——总参数可以很大（如 DeepSeek V3 的 671B），但每次推理的激活参数很少（37B）。

劣势：① 训练更不稳定（负载均衡是核心难题），② 专家切换可能引入推理断裂，③ 全量推理时显存占用极大（所有 expert 都要在 GPU 里）。

用 MoE 还是 Dense，取决于"你有多少显存预算"和"你的任务是结构化还是模糊的"。

Q8: FlashAttention 为什么能加速？算法变了吗?

算法**完全不变**——输出结果和原始 attention 完全一样，精确到 bit。加速来自 IO 优化。

GPU 有两层存储：HBM（大但慢，QKV 和中间结果全在这）和 SRAM（小但快 10 倍）。原始 attention 反复把 QKV 从 HBM 搬到 SRAM、算出中间结果搬回去、再搬回来做 softmax——80% 时间浪费在搬运上。

FlashAttention 用 **tiling**：把 QKV 切成小块，每次搬一小块到 SRAM 里一口气算完，大大减少了 HBM 往返次数。用 **online softmax** 解决了分块 softmax 需要全局 max 的难题。

简单说：计算顺序换了，算法和结果不变，但 GPU 的空转时间大幅减少。

Q9: KV Cache 是什么？推理时显存为什么会炸？

KV Cache = 把之前 token 的 Key 和 Value 矩阵存下来，新 token 进来时只算新 token 的，之前的不重复算。

显存为什么炸？ 假设 LLaMA 70B：80 层、8 个 KV head、128 维。4K token 的 KV cache 约 10GB。8K 约 20GB。加上模型参数本身（140GB FP16），一张 H100 80GB 就爆了。这就是为什么所有推理优化都在打 KV cache 的主意——缩减它不是为了让模型更快，是让它**根本跑得动**。

Q10: MHA → MQA → GQA → MLA 这条演进线说明了什么？

这是一条 KV Cache 效率从粗放到精细的演进：

演进	核心做法	KV Cache 缩减	代价
MHA (2017)	每个 attention head 独立 K,V	基线	最大
MQA (2019)	所有 head 共享一套 K,V	1/h	精度损失最大
GQA (2023)	分组共享（如 8 头分 2 组）	1/g	精度损失小
MLA (2024)	K,V 低秩分解到潜空间	~10x	工程复杂度高

这条线说明了一件事：**多头之间的 K、V 大量冗余**——8 个 head 算出来的 K 矩阵其实高度相关，没必要独立存 8 份。从粗暴共享到精细压缩，注意力设计在逼近一个理论界限：用最少的 bit 存下最多的"有效信息"。

场景题

场景 1：1M Context Window 的 Transformer 怎么设计？

> 你的老板说："竞争对手新模型支持 1M 上下文了，我们也要跟上。"你从哪些维度考虑方案？

答题框架：

1. **Attention 选型**：MHA 扛不住 1M。上 GQA（至少 4 组）或 MLA。或者混合架构——早期层用稀疏注意力/滑动窗口，只在高层的几个关键层做全局注意力。
1. **位置编码**：RoPE 需要调整 base frequency（增大到 500,000–1,000,000）才能外推到 1M。或者上 YaRN、NTK-aware scaling 这类专门的外推方案。
1. **FlashAttention 必备**：1M 的 QK^T 矩阵是 1M×1M，$O(n^2)$ 不优化跑不了。
1. **Ring Attention / 序列并行**：1M context 一个 GPU 放不下，需要对序列维度做并行切分（不是数据并行也不是模型并行），多个 GPU 各算一部分注意力。
1. **KV Cache 策略**：1M context 的 KV cache 差不多要几百 GB——必须用量化（INT8/INT4 KV cache）+ 分页管理（vLLM 的 PagedAttention）+ 选择性丢弃。
1. **评估**：定了方案后，先跑 RULER、LongBench 这类长上下文 benchmark，确认检索精度不随位置衰减。

场景 2：1B 参数预算，自回归语言模型，选什么架构？

> 你有个 1B 参数的预算，要训一个通用语言模型。架构怎么选?

分析：

1. 1B 参数在这个时代算「小而美」。MoE 没必要——总参数量太小，拆开 expert 每个只有几百 M，专业化优势不明显，路由开销反而占了大头。
1. **Decoder-only Dense Transformer**。标准组件：RoPE + GQA + SwiGLU + RMSNorm + Pre-Norm。参考 Qwen2.5-0.5B 和 1.5B 之间的架构——不搞花活，吃透 Scaling Laws。
1. 数据比架构重要：按 Chinchilla ~20 tokens/参数，需要 ~20B 高质量 tokens。如果有能力，过训练（50B+ tokens）会更好——LLaMA 3 8B 用了 ~1,875 tokens/参数。
1. 上下文长度：1B 参数不需要 128K，32K 够了，但位置编码用 RoPE（方便后期继续训练扩展）。
1. 推理部署：1B 参数能直接跑在手机上（量化到 INT4 约 500MB），这是实际落地的关键优势。

场景 3：低延迟在线客服系统的模型选型

> 你要为一个电商平台部署实时在线客服。要求：首 token 延迟 <300ms，支持流式输出，日均 10 万次对话。

分析：

1. **Dense 小模型 > MoE 大模型**。在线客服的最低延迟要求（<300ms TTFT）基本排除了大模型。7B-13B 参数的 Dense 模型，用 vLLM + prefix caching 可以达到 <100ms TTFT。
1. **Speculative Decoding**：用 0.5B draft model + 7B target model，2-3x 推理加速而不损失质量。
1. **KV Cache 优化**：GQA 或 MQA 减少 KV cache = 更大的 batch = 更高吞吐。PagedAttention 管理内存碎片。
1. **System Prompt 缓存**：客服的 system prompt 每次都一样，vLLM 的 Automatic Prefix Caching 可以直接复用，省去每次重新计算 K、V。
1. **量化**：INT4/INT8 把模型压缩到 4-8GB，能在一张 A10/L40 上服务更多并发。INT4 精度损失 <1%，对于客服场景几乎无感。

场景 4：模型训练 Loss 不下降，怎么排查?

> 你在训一个小 Transformer，loss 卡住不降了。

排查清单（按优先级）：

1. **Learning Rate 太大** → NaN loss? lr 降到 1/10 再试
2. **Learning Rate 太小** → loss 降得极慢? warmup 太小或 lr 峰值不够
3. **梯度爆炸** → loss 突然蹦到 NaN? 加 gradient clipping（1.0-5.0）
4. **数据问题** → 检查几个 batch 的实际输入输出，tokenizer 是不是对的
5. **注意力 mask 搞反了** → 因果 mask 应该是上三角 $-\infty$，不是你设的下三角
6. **Pre-LN vs Post-LN** → 深层 (>20 层) 必须用 Pre-LN

7. **Label Smoothing →** 如果太过，loss 降得很慢看着像"天花板"（0.1 通常安全）
8. **Adam β 参数 →** β1=0.9 β2=0.95–0.999 是主流。β2 太小 = 不稳定

场景 5：你的团队要在 Dense 和 MoE 之间做选择，你怎么建议？

决策框架：

条件	建议
参赛规模 <10B	Dense
参赛规模 10–50B	Dense 或 small MoE (4–8 experts)
参赛规模 >50B	MoE（除特殊需求外）
推理吞吐是最核心指标	Dense（省 KV cache、省 expert loading 开销）
推理成本是最核心指标	MoE（激活参数小、快）
任务需要长链深度推理	Dense（一致性顾虑）
需要在边缘设备运行	Dense（MoE 的全量专家加载不适合内存受限场景）
团队训练经验不足	Dense（MoE 训练调参更复杂）

一句话：**大参数 + 低成本 = MoE，小参数 + 简单可靠 = Dense。**

应用题

应用 1：Transformer 在新闻摘要系统中的应用

> 用 Transformer 搭建一个每天处理 500 篇新闻的自动摘要系统。

架构设计：

- 用 Encoder–Decoder Transformer（BART/T5 系在摘要任务上天然更适合），或 Decoder–only + prompt template
- 长文档需要切分 → 分块分别摘要 → 再摘要几个摘要（层级摘要法）
- 结合本文项目的实际经验：用 HanLP 做中文分句、用 Claude API 做标题翻译、用 content_hash 做去重

实际流水线：采集 → 去重 → 分块 → 逐块编码 → 解码生成摘要 → 后处理 → 分发

应用 2：Transformer 在代码补全工具中的应用

> GitHub Copilot 这种代码补全工具怎么用 Transformer?

关键设计：

- Decoder-only 模型（代码续写也是自回归生成），用 Fill-in-the-Middle（FIM）训练格式
- FIM 格式（以 StarCoder 为例）：`<PRE> prefix code <SUF> suffix code <MID> completion code`
- 上下文窗口管理：代码文件可能很长，但补全时只看光标附近的几百行和 import 区
- 延迟硬要求：<200ms TTFT，意味着模型不能太大（1-3B 参数为主），大量使用 speculative decoding 和 KV cache 共享

应用 3：Transformer 在多模态搜索中的应用

> 怎么用 Transformer 做一个"传一张照片搜相似商品"的功能?

架构：CLIP 风格的双塔模型

- 图像塔：ViT 编码图片 → 一个向量
- 文本塔：Transformer 编码商品描述 → 一个向量
- 对比学习（Contrastive Loss）让匹配的图文对向量接近、不匹配的远离
- 推理时：图片向量去向量数据库做 ANN 搜索，召回 top-K 最相似商品描述

为什么会用 Transformer 而不是 CNN 给图像编码? ViT 把图像切成 16x16 的 patch，每个 patch 当成一个 token，用 Transformer 建模 patch 之间的关系——全局视野比 CNN 的局部卷积更完整。2020 年以来的实践反复证明 Transformer 的特征提取能力在不同的模态上都通用——这就是为什么"都上 Transformer"是合理的默认策略。

七、信息来源

核心论文

- Vaswani et al., "Attention Is All You Need" (2017) — https://arxiv.org/abs/1706.03762
- Bahdanau et al., "Neural Machine Translation by Jointly Learning to Align and Translate" (2014) — https://arxiv.org/abs/1409.0473
- Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers" (2018) — https://arxiv.org/abs/1810.04805
- Radford et al., "Language Models are Unsupervised Multitask Learners" (GPT-2, 2019)
- Brown et al., "Language Models are Few-Shot Learners" (GPT-3, 2020) — https://arxiv.org/abs/2005.14165
- Kaplan et al., "Scaling Laws for Neural Language Models" (2020) — https://arxiv.org/abs/2001.08361
- Hoffmann et al., "Training Compute-Optimal Large Language Models" (Chinchilla, 2022) — https://arxiv.org/abs/2203.15556
- Chowdhery et al., "PaLM: Scaling Language Modeling with Pathways" (2022) — https://arxiv.org/abs/2204.02311

- Touvron et al., "LLaMA: Open and Efficient Foundation Language Models" (2023) — https://arxiv.org/abs/2302.13971
- Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention" (2022) — https://arxiv.org/abs/2205.14135
- Dao et al., "FlashAttention-2: Faster Attention with Better Parallelism" (2023) — https://arxiv.org/abs/2307.08691
- Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision" (2024) — https://arxiv.org/abs/2407.08614

MoE 相关

- Shazeer et al., "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer" (2017) — https://arxiv.org/abs/1701.06538
- Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation" (2020) — https://arxiv.org/abs/2006.16668
- Fedus et al., "Switch Transformers: Scaling to Trillion Parameter Models" (2021) — https://arxiv.org/abs/2101.03961
- DeepSeek, "DeepSeekMoE: Towards Ultimate Expert Specialization" (2024.01) — https://arxiv.org/abs/2401.06066
- DeepSeek, "DeepSeek-V2: A Strong, Economical, and Efficient MoE Language Model" (2024.05) — https://arxiv.org/abs/2405.04434
- DeepSeek, "DeepSeek-V3 Technical Report" (2024.12) — https://arxiv.org/abs/2412.19437

位置编码 / 注意力变体

- Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021) — https://arxiv.org/abs/2104.09864
- Press et al., "Train Short, Test Long: Attention with Linear Biases (ALiBi)" (2021) — https://arxiv.org/abs/2108.12409
- Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need" (MQA, 2019) — https://arxiv.org/abs/1911.02150
- Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models" (2023) — https://arxiv.org/abs/2305.13245

替代架构

- Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces" (2023) — https://arxiv.org/abs/2312.00752
- Gu & Dao, "Transformers are SSMs" (Mamba-2, 2024) — https://arxiv.org/abs/2405.21060

分析报道

- "The Evolution of FlashAttention" — ICLR Blog (2026) — https://iclr-blogposts.github.io/2026/blog/2026/the-evolution-of-flashattention/
- "The Big LLM Architecture Comparison" — https://www.notion.so/hismayfly/The-Big-LLM-Architecture-Comparison
- "The DeepSeek Series: A Technical Overview" — Martin Fowler — https://martinfowler.com/articles/deepseek-papers.html
- "8种LLM架构设计大比拼" — 51CTO (2026)
- Geva et al., "Transformer Feed-Forward Layers Are Key-Value Memories" (2021) — https://arxiv.org/abs/2012.14913

> **方法论说明**：本报告使用横纵分析法（Horizontal-Vertical Analysis），由数字生命卡兹克提出。纵向追时间深度（从诞生到当下的完整发展历程），横向追同期广度（当前竞争格局的全面对比），最终在交叉处产出新的判断。

>

> 写作参考了 MoE 分析报告的叙事风格——以具体事件和决策为锚点，在技术细节与行业叙事之间寻找平衡，避免咨询报告式的空洞概括。